

I2236 INTRODUCTION TO EMBEDDED SYSTEMS

LECTURE 4 – PART 2 INTERFACING PERIPHERALS

Bilal Daass
04/02/2026



COURSE OVERVIEW

Module 1: Introduction & Foundations (Week 1–2)

Module 2: Arduino Ecosystem & Programming Basics (Week 3–4)

Module 3: Digital & Analog Interfacing (Week 5–6)

Module 4: Interfacing Peripherals (Week 7–8)

Module 5: Sensors & Actuators (Week 9–10)

Module 6: Communication & Integration (Week 11–12)

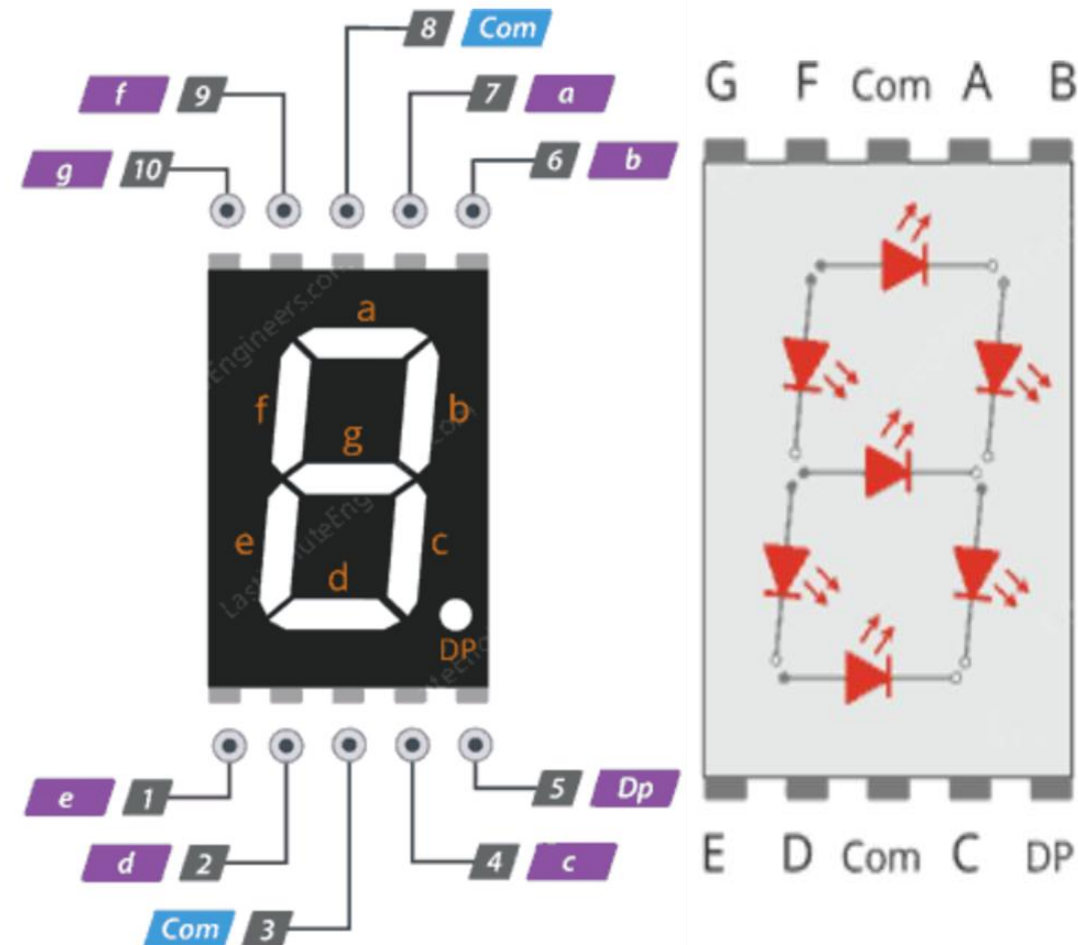
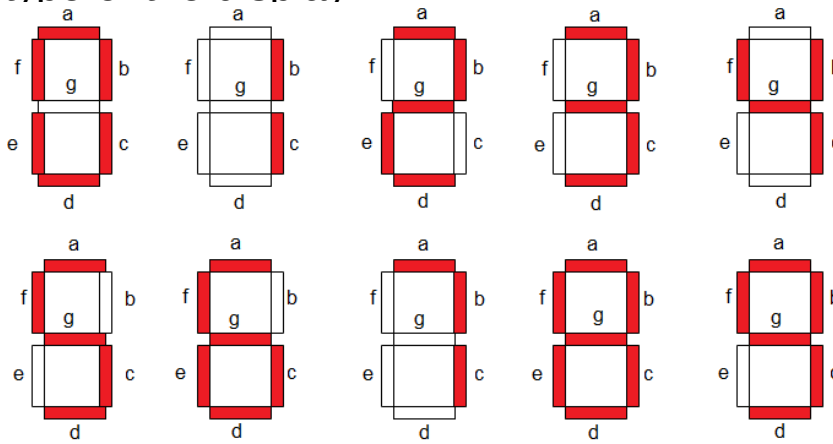
Module 7: Project (Week 13–15)

- **Input devices:**
 - Keypad,
 - Ultrasonic Distance Sensor,
 - Passive infrared sensor (PIR),
 - Other sensors (Humidity and temperature sensor, Hall effect sensor, Accelerometer and gyroscope).
- **Output devices:**
 - 7-segment display,
 - Servo motor,
 - LCD,
 - IR Remote.

7-SEGMENT DISPLAY

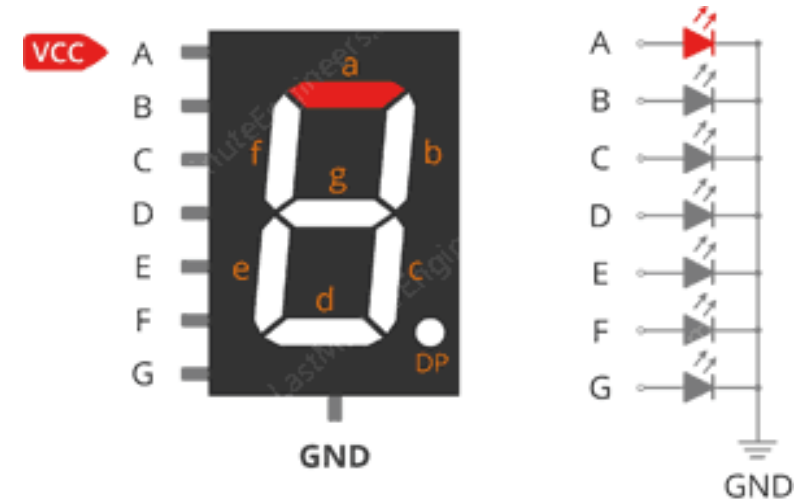
7-SEGMENT DISPLAY

- The 7-segment displays are used to display numbers and characters. They are seven LEDs lined up in a particular pattern. Each of the seven LEDs is called a segment.
- An additional 8th LED is used for indication of a decimal point.
- a-g & DP segment (decimal point) are connected to digital pins of Arduino. By controlling each LED on the segment connected, numbers can be displayed.
- COM The pin 3 and 8 are internally connected to form a common pin. This pin should be connected to GND (**common cathode**) or 5V (**common anode**) depending upon the type of the display.

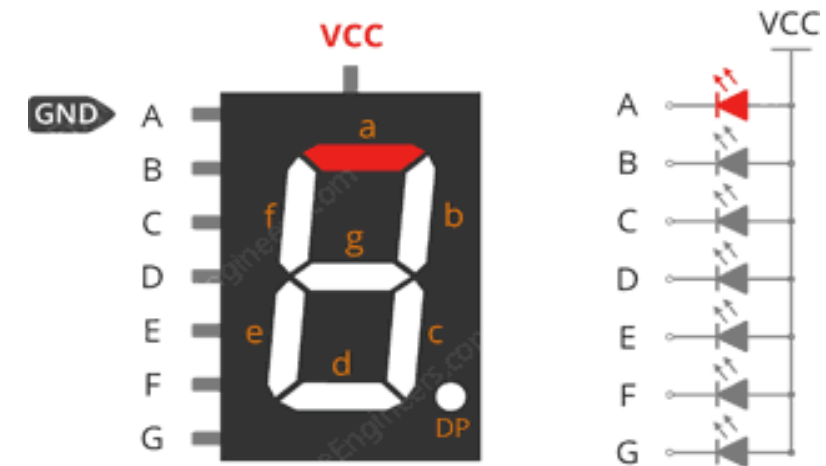


7-SEGMENT DISPLAY

- Seven segment displays are of two types: **Common Cathode (CC)** and **Common Anode (CA)**.
 - The difference is the polarity of the LEDs and common terminal.
 - The common cathode has all the cathodes of the LEDs in a 7-segment connected together and the common anode has all the anodes of the LEDs in a 7-segment connected together.
- In the common cathode display, all the cathode connections of the LED segments are connected together to 'logic 0' / GND.
 - The individual segments are then illuminated by applying HIGH / 'logic 1' signal to the individual Anode terminals (a-g).
- In the common anode display, all the anode connections of the LED segments are joined together to logic "1".
 - The individual segments are illuminated by applying a ground, logic "0" or "LOW" signal to the Cathode of the particular segment (a-g).



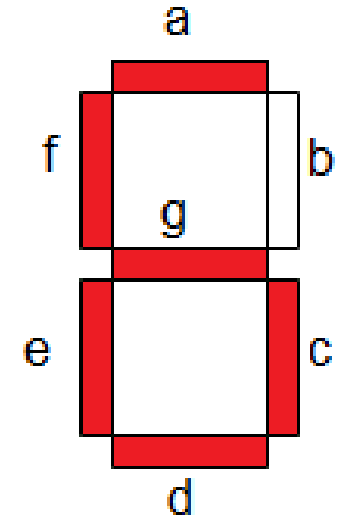
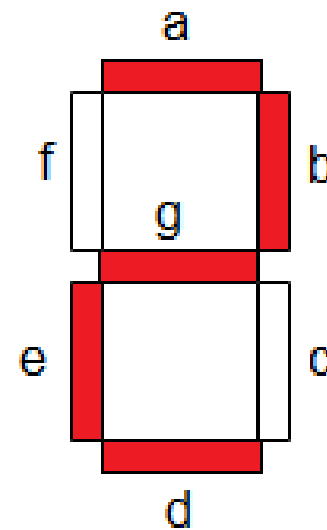
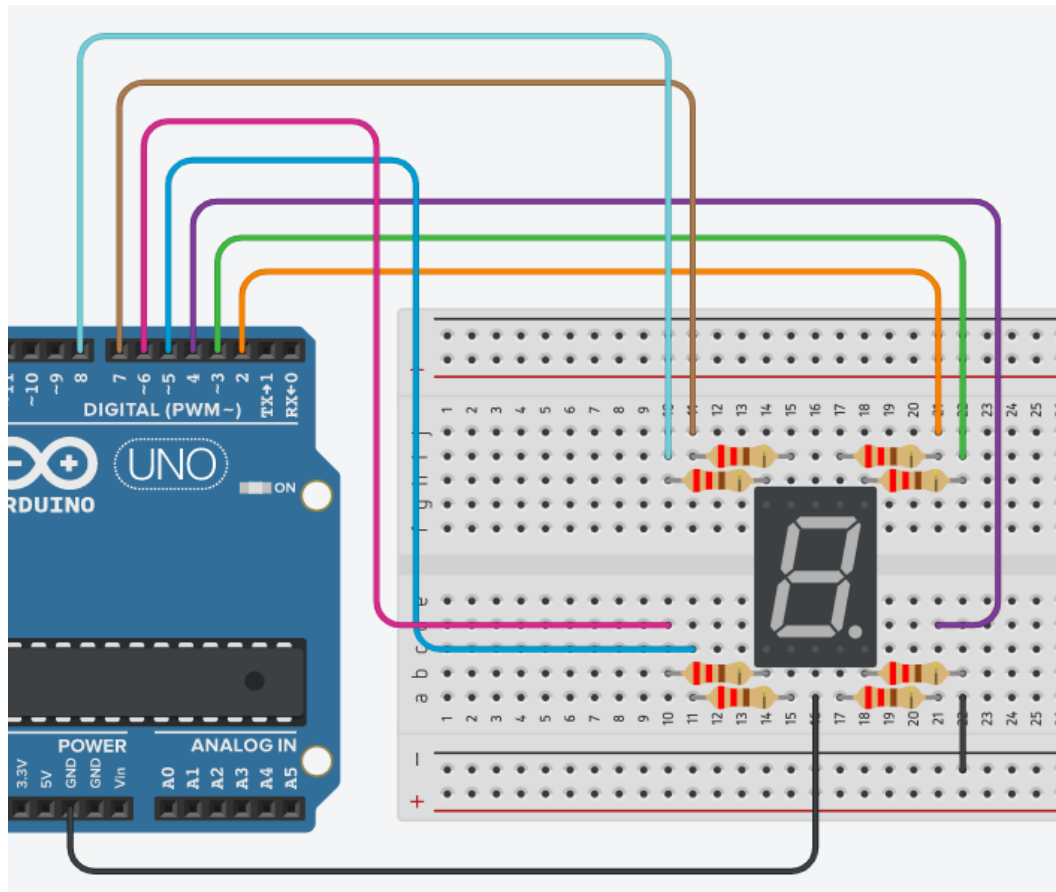
Common cathode



Common anode

7-SEGMENT DISPLAY

Example : Write a code that alternately displays the numbers *two* and *six*.



7-SEGMENT DISPLAY

```
int A = 2;
int B = 3;
int C = 4;
int D = 5;
int E = 6;
int F = 7;
int G = 8;

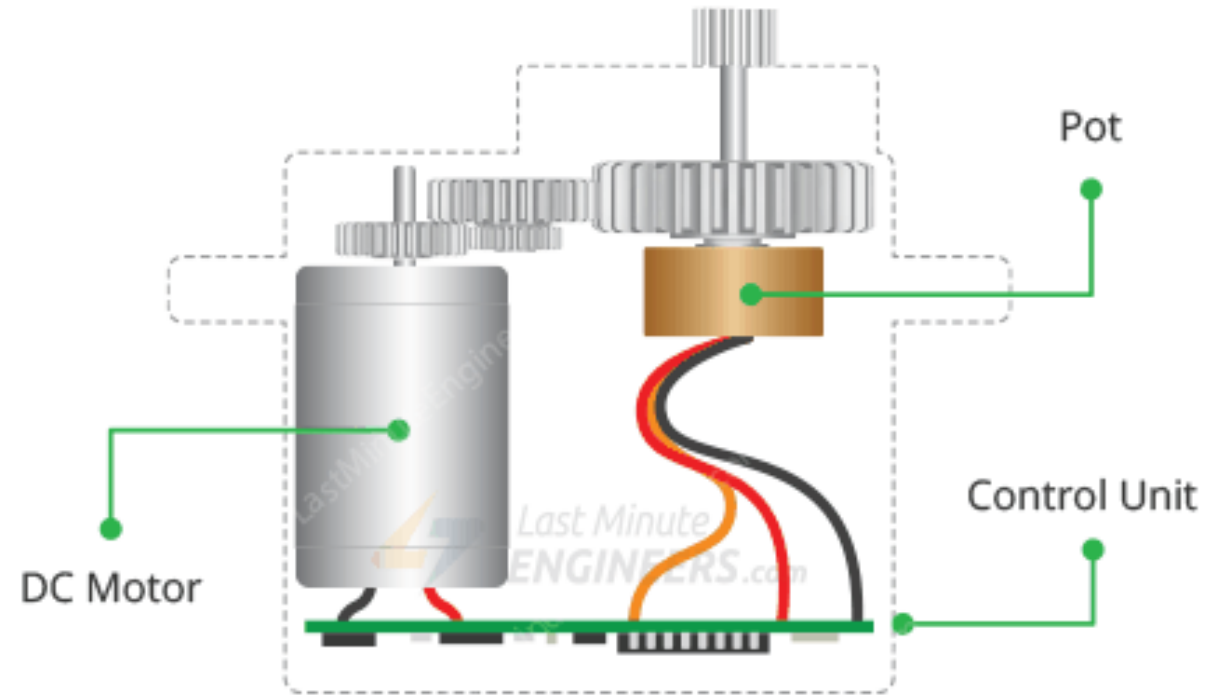
void setup(void)
{
    pinMode(A, OUTPUT);
    pinMode(B, OUTPUT);
    pinMode(C, OUTPUT);
    pinMode(D, OUTPUT);
    pinMode(E, OUTPUT);
    pinMode(F, OUTPUT);
    pinMode(G, OUTPUT);
}
```

```
void loop(void)
{ // display 2
    digitalWrite(A, HIGH);
    digitalWrite(B, HIGH);
    digitalWrite(C, LOW);
    digitalWrite(D, HIGH);
    digitalWrite(E, HIGH);
    digitalWrite(F, LOW);
    digitalWrite(G, HIGH);
    delay(1000);
    // display 6
    digitalWrite(A, HIGH);
    digitalWrite(B, LOW);
    digitalWrite(C, HIGH);
    digitalWrite(D, HIGH);
    digitalWrite(E, HIGH);
    digitalWrite(F, HIGH);
    digitalWrite(G, HIGH);
    delay(1000);
}
```


SERVO MOTOR

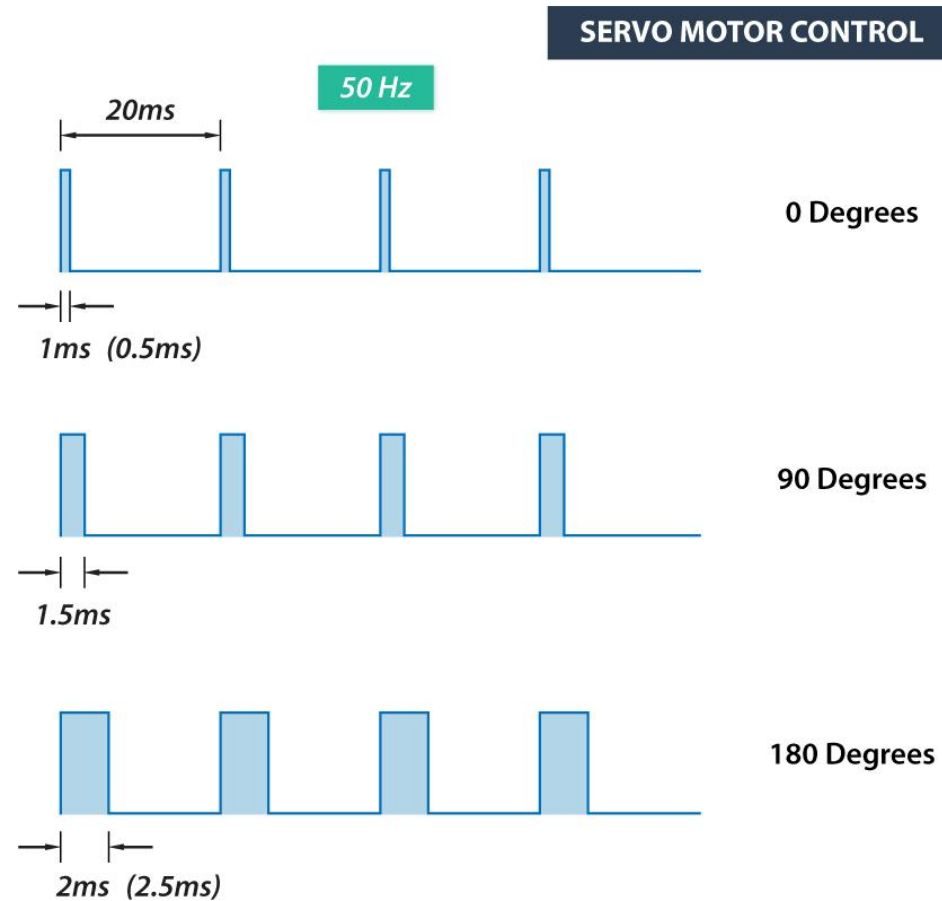
SERVO MOTOR

- Servo motor is a low-speed, high-torque motor that can turn to a specified position.
- Servo motor contains a small DC motor connected to the output shaft through the gears.
- The potentiometer provides position feedback to the servo control unit where the current position of the motor is compared to the target position.
- According to the error, the control unit corrects the actual position of the motor so that it matches the target position.
- Servos have integrated gears and a shaft that can be precisely controlled.
- A control signal is sent to the servo to position the shaft at the desired angle.
- It can be positioned at various angles, usually between 0 and 180 degrees. It does not spin 360 degrees.



SERVO MOTOR

- The control signal is a pulse width modulated (PWM) signal
- A conventional analog servo motor expects to receive a pulse roughly every 20 milliseconds (i.e. signal should be 50Hz).
- The length of the pulse determines the position of the servo motor:
- If the pulse is high for 1ms, then the servo angle will be zero.
- If the pulse is high for 1.5ms, then the servo will be at its center position.
- Pulses ranging between 1ms and 2ms will move the servo shaft to a position between 0 and 180 degrees.

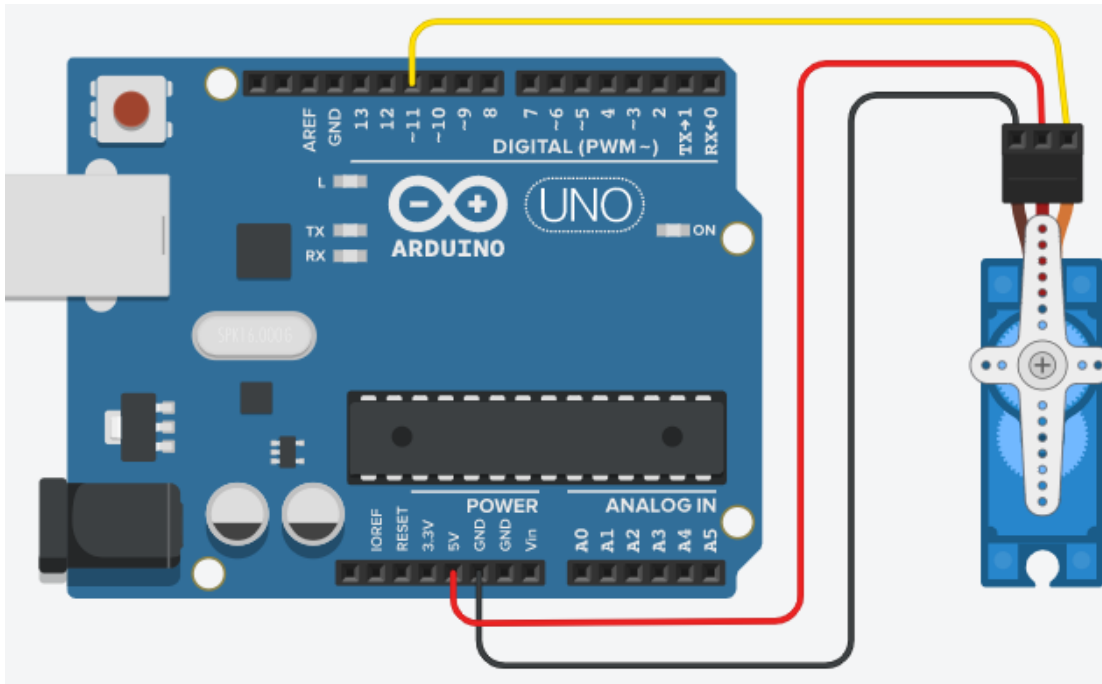


SERVO MOTOR

Servo motor connections

The servo motor has three connections:

- red for power,
- brown or black for ground
- and orange or yellow for control signal



- Motors requiring high current should not be powered directly by the Arduino : An external power source should be used.
- For Arduino UNO, the maximum current per IO pin is 40 mA.
- The maximum total current from all the IO pins together is 200 mA.
- SG90 Micro Servo motor runs on 4.8-6VDC (5V Typical). It consumes around 10mA at idle and 100mA to 250mA when moving, so we can power it up through 5-volt pin on the Arduino board.
- If you have a servo that consumes more than 250mA, consider using a separate power supply for your servo.

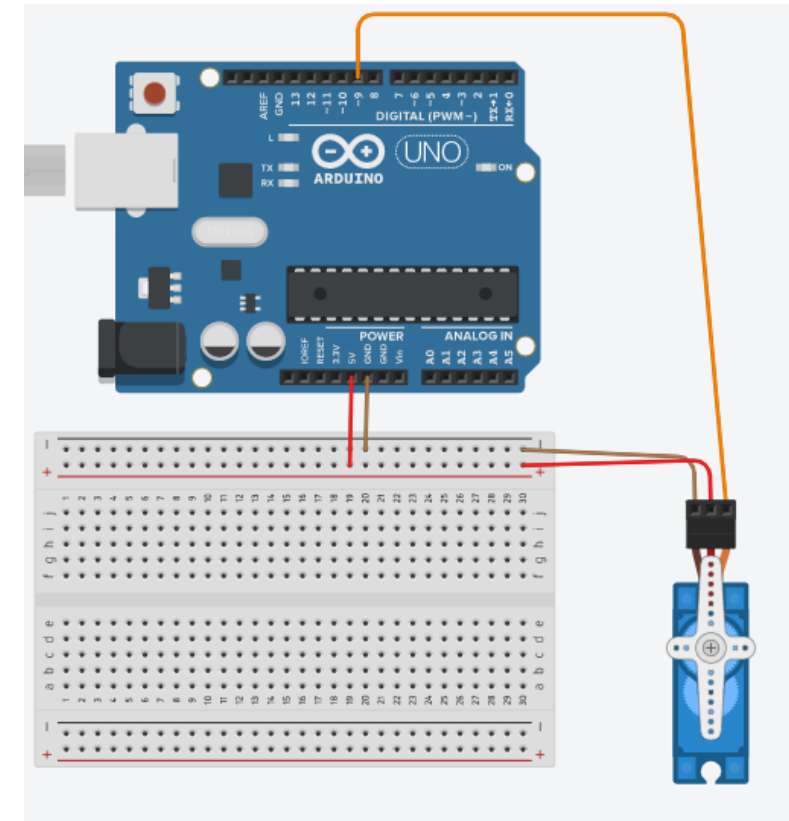
SERVO MOTOR

Simple application

```
#include <Servo.h>
Servo servo; // associate servo with Servo library
int servoPin = 9; // servo motor pin

void setup()
{
    servo.attach(servoPin); // define servo motor pin to Servo library
}

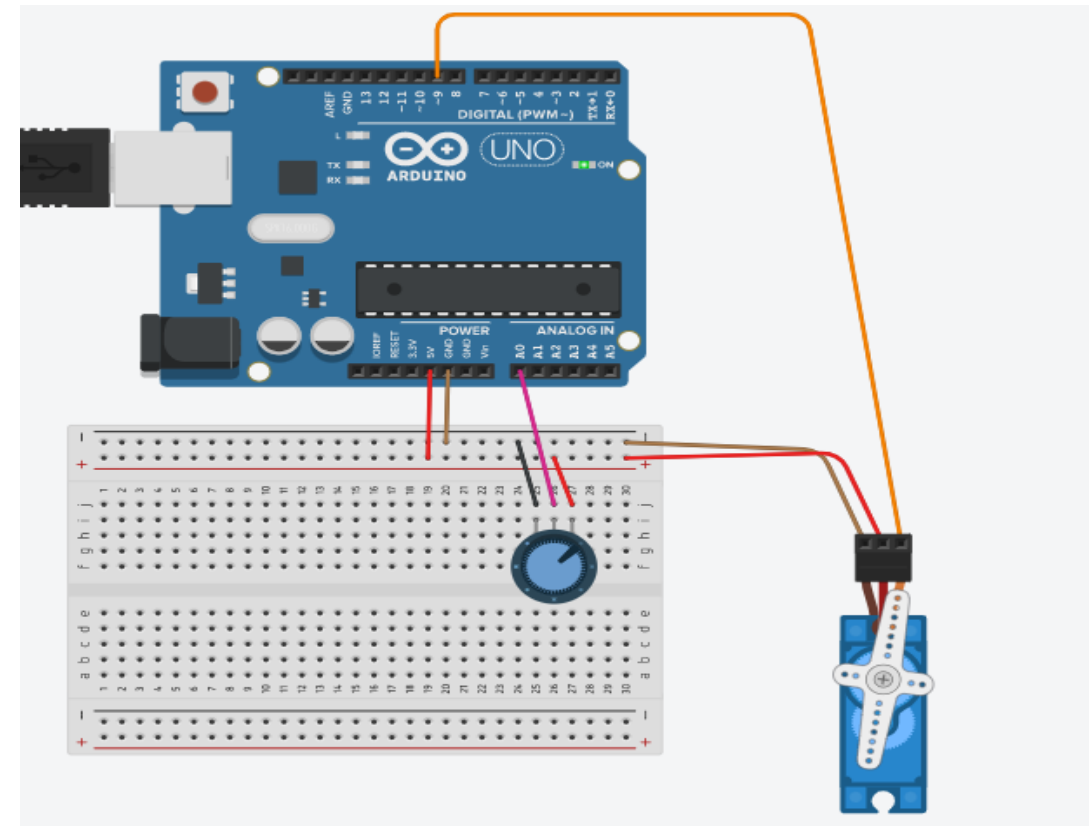
void loop()
{
    for (int angle=0; angle<=180; angle++)
    {
        servo.write(angle); // rotate to angles 0, 1, 2 ... 180
        delay(50); // delay 50ms between movements
    }
    for (int angle=180; angle>=0; angle--)
    {
        servo.write(angle); // rotate to angles 180, 179 ... 0
        delay(25); // delay 25ms between movements
    }
}
```



SERVO MOTOR

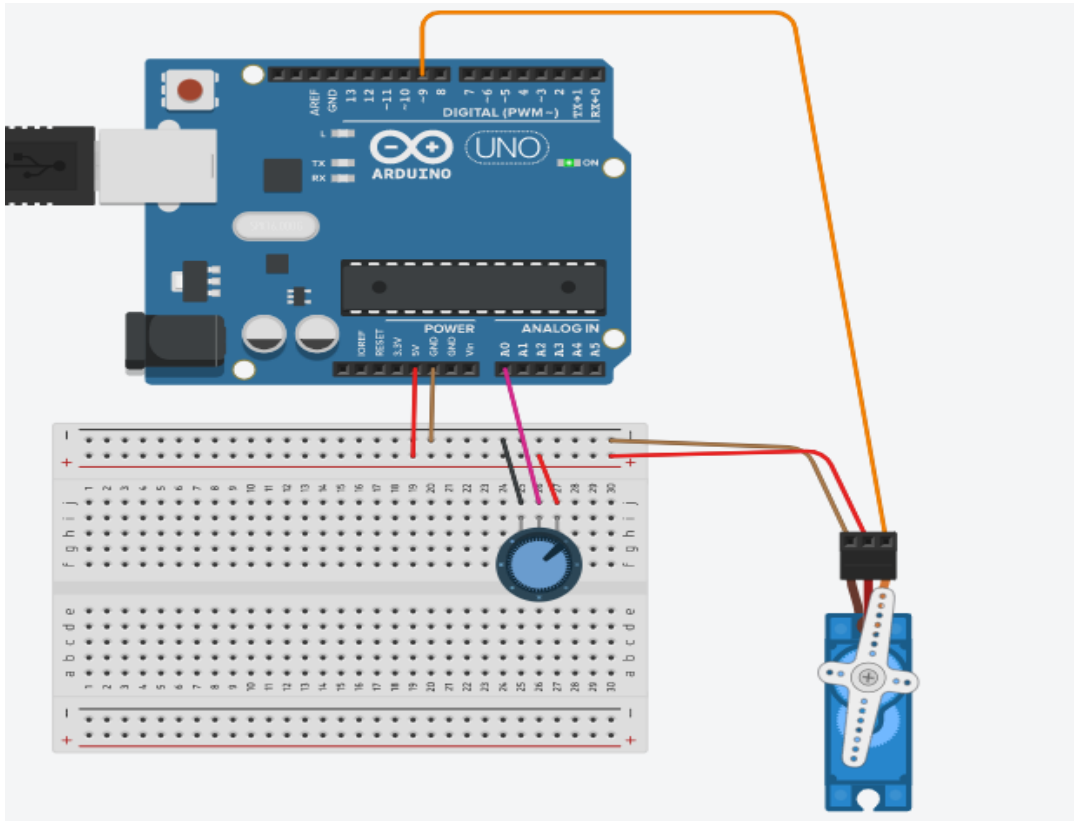
Controlling Servo with a Potentiometer

Control the rotation angle of a servo motor using a potentiometer.



SERVO MOTOR

Controlling Servo with a Potentiometer



```
#include <Servo.h>
```

```
Servo servo;
```

```
int servoPin = 9;
```

```
int reading;
```

```
int potPin = A0;
```

```
int angle;
```

```
void setup()
```

```
{
```

```
    servo.attach(servoPin);
```

```
}
```

```
void loop()
```

```
{
```

```
    reading = analogRead(A0);
```

```
    angle = map(reading, 0, 1023, 0, 180);
```

```
    servo.write(angle);
```

```
    delay(50); // delay 50ms between movements
```

```
}
```

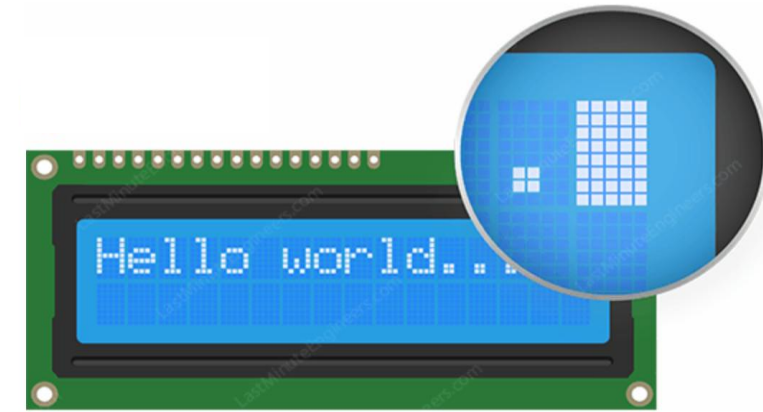
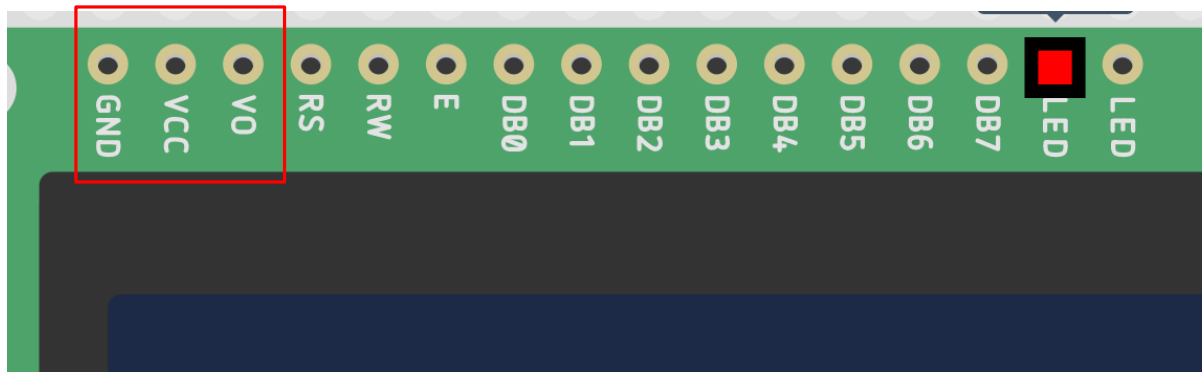
LCD

LCD

- Liquid Crystal Display (LCD) are common to displays output from the Arduino.
- LCDs have different sizes and colors (for example 16×2, 16×4, 20×2, and 20×4 with white text on blue background, with black text on green, ...)
- LCDs have little rectangles for each character on the display and the pixels that make up a character. Each of these rectangles is a grid of 5×8 pixels.

Connections

1. **GND** should be connected to the ground
2. **VCC** is the power supply for the LCD and should be connected to 5 volts
3. **Vo** (LCD Contrast) controls the contrast of the displayed: The contrast can be adjusted using a voltage divider with a potentiometer

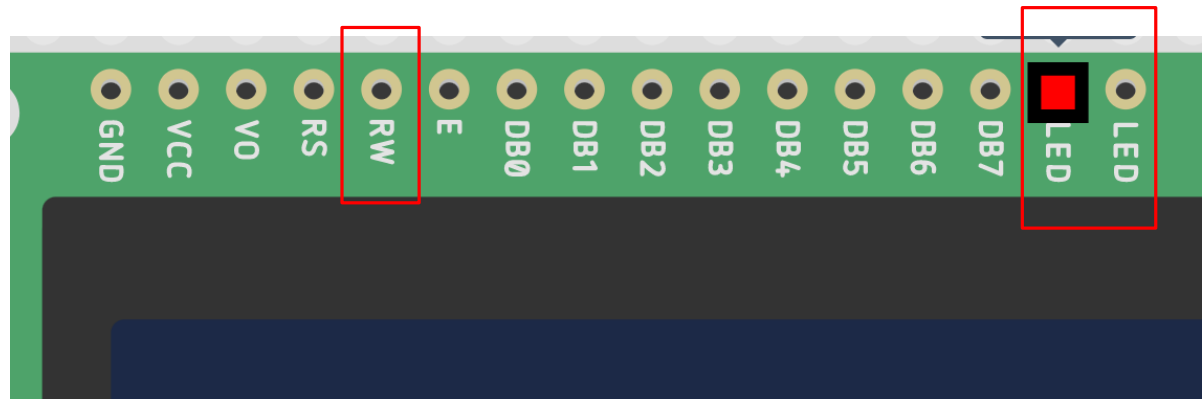


16×2 LCD

LCD

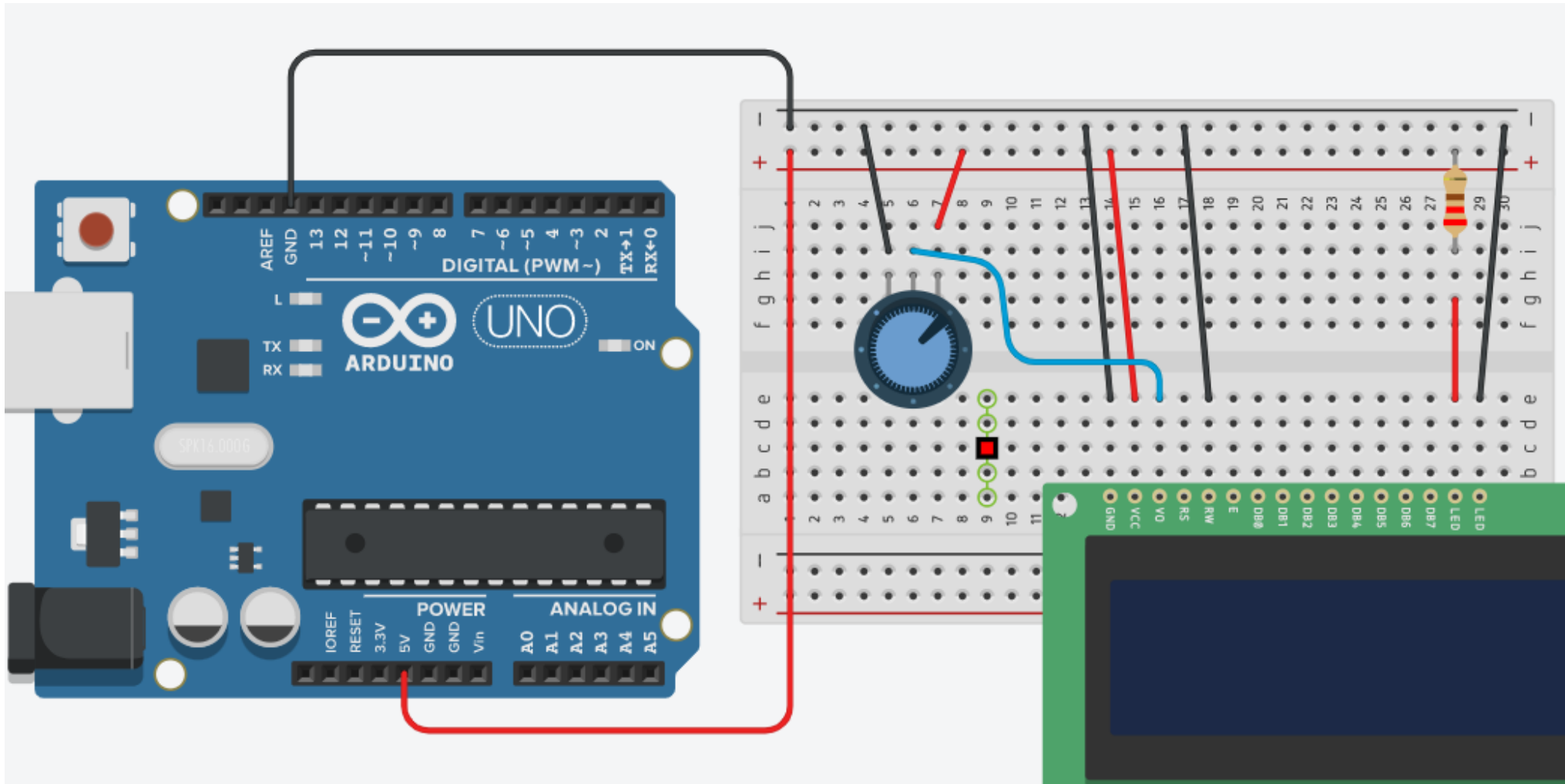
Connections

4. RW (Read/Write) pin controls whether you are reading data from the LCD or writing data to it. Since we're just using this LCD as an OUTPUT device, we're going to tie this pin to LOW (GND). This forces it into the WRITE mode.
5. Anode & Cathode pins are used to control the backlight LED of the LCD. They can be connected to 5 volts and GND, in series with 220 Ohms resistor



LCD

Connections



LCD

Connections

6. *E (Enable) pin is used to enable the display*
7. *D0-D7 are the pins that carry the 8 bit data we send to the display. But, HD44780 LCDs are designed in a way that we can talk to the LCD using only 4 data pins(4-bit mode) instead of 8(8-bit mode).*

They will be connected to digital pins:

LCD RS pin to digital pin 12

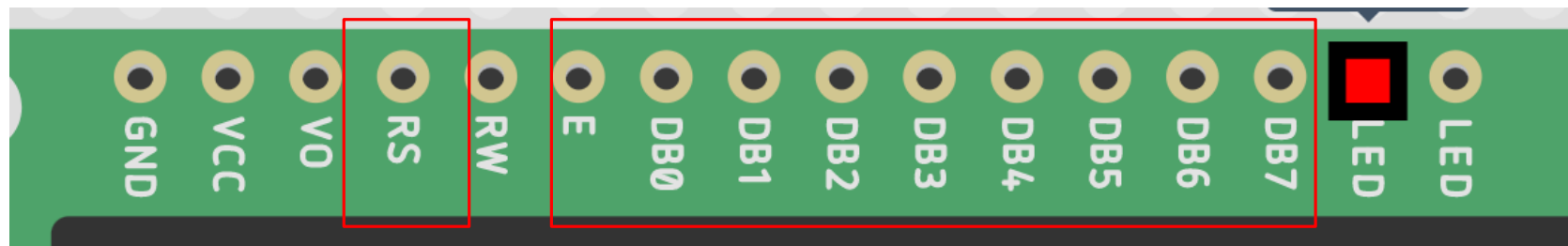
LCD E pin to digital pin 11

LCD D4 pin to digital pin 5

LCD D5 pin to digital pin 4

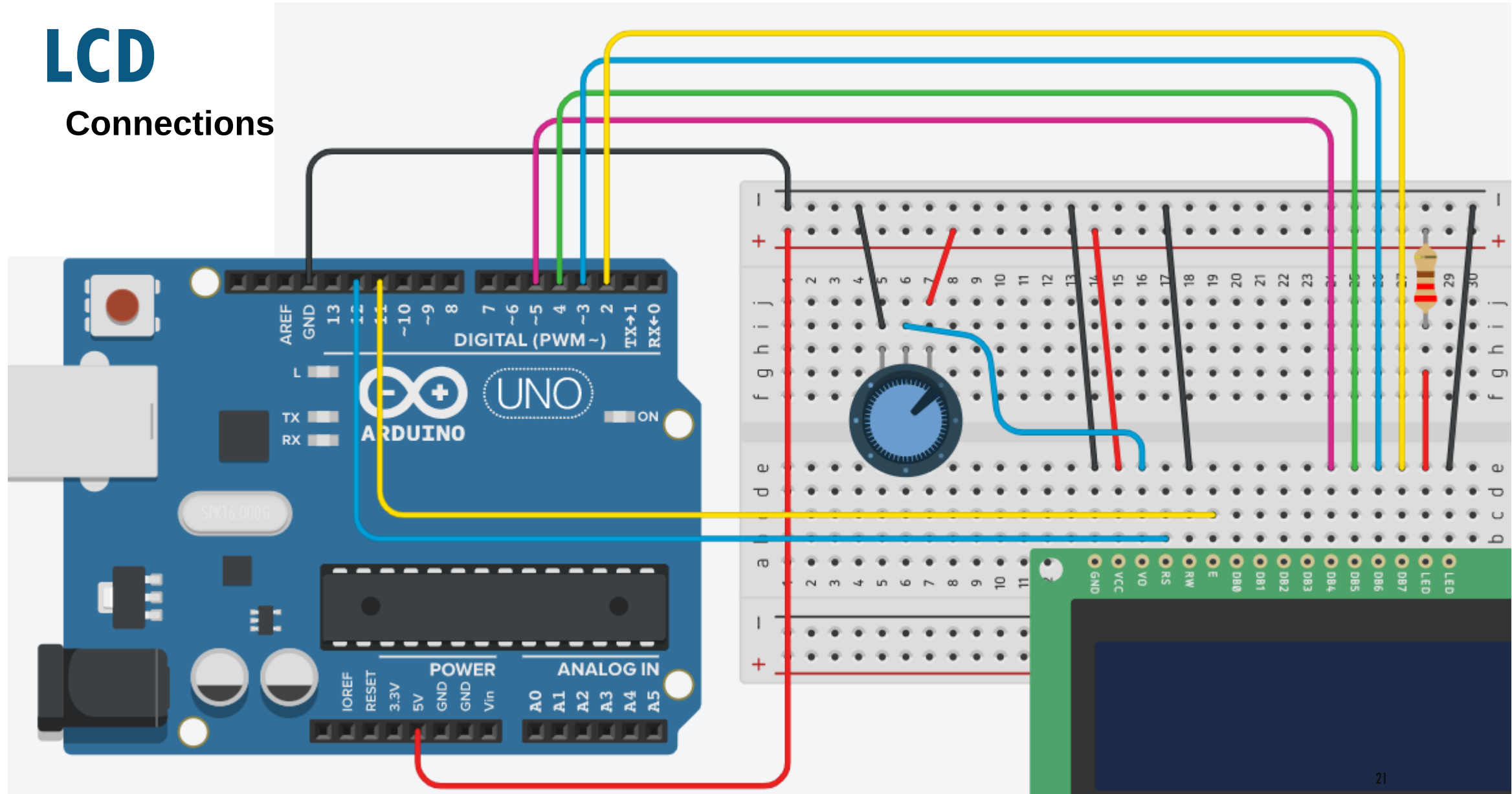
LCD D6 pin to digital pin 3

LCD D7 pin to digital pin 2



LCD

Connections

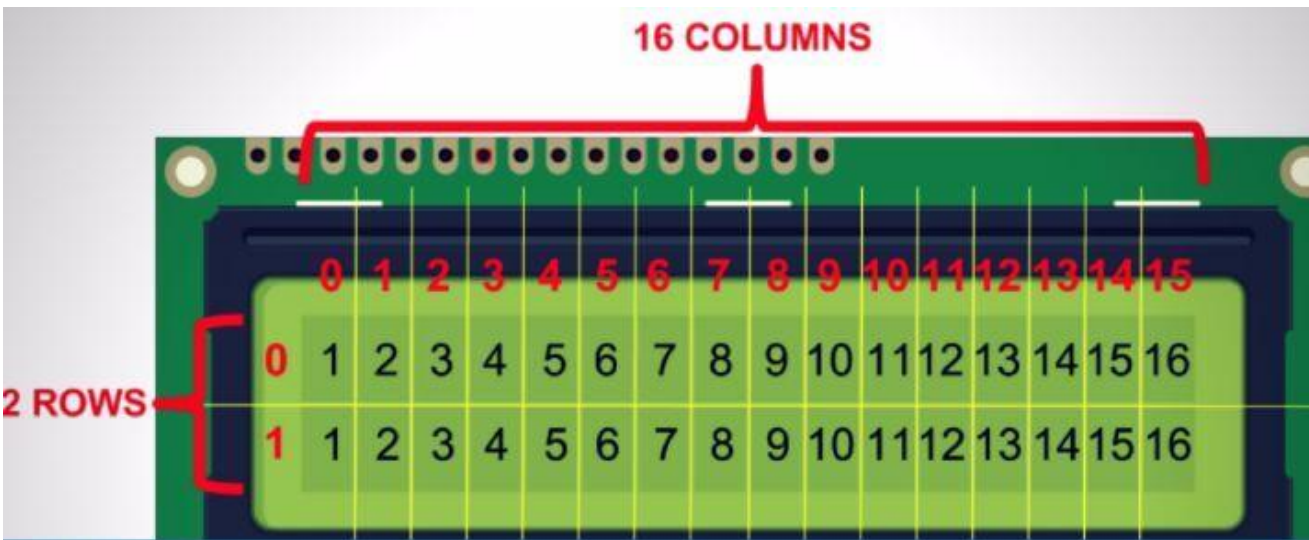


LCD

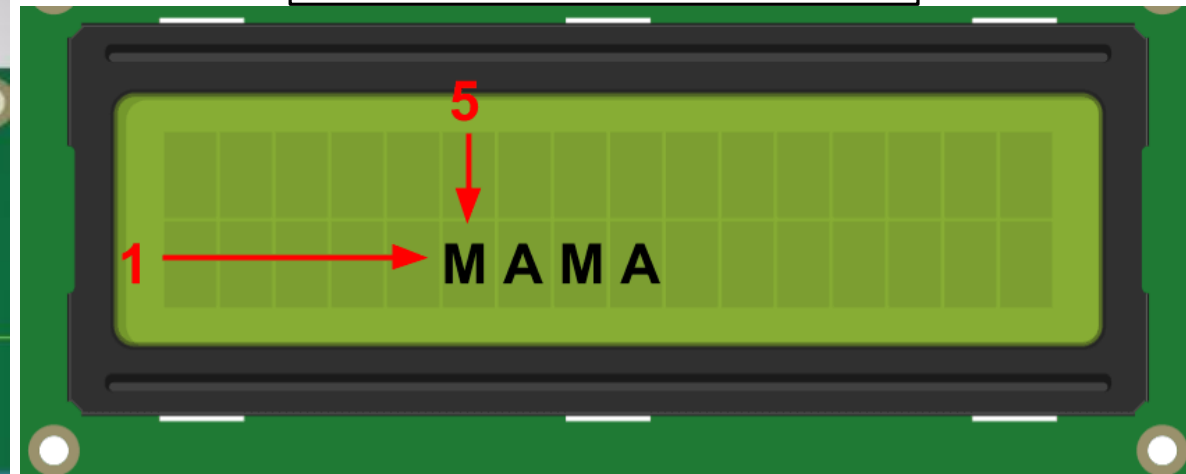
Code

1. set the location at which text written to the LCD will be displayed.

```
lcd.setCursor(column, row);
```



```
lcd.setCursor(5, 1);  
lcd.print("MAMA");
```



LCD

Code – Other functions

- **To delete a character:**
 - Place the cursor at the corresponding position using `lcd.setCursor(..., ...)`
 - Then execute `lcd.print(" ")` // print a white space
- `lcd.clear()` : **delete everything** // Clears the LCD screen and positions the cursor in the upper-left corner.
- `lcd.home()` : positions the cursor in the upper-left of the LCD without clearing the display. It's equivalent to `lcd.setCursor(0,0)`
- `lcd.blink()` : displays a blinking block at the position to which the next character will be written
- `lcd.cursor()` : displays an underscore (line) at the position to which the next character will be written
- You can use the `lcd.noBlink()` function to turn off the blinking LCD cursor and `lcd.noCursor()` to hide the LCD cursor.
- You can scroll the contents of the display one space to the right using `lcd.scrollDisplayRight()` or one space left using `lcd.scrollDisplayLeft()`

LCD

Code

```
// include is used to include outside libraries in your sketch.
#include <LiquidCrystal.h>
// Create an LCD object. Parameters: (rs, en, d4, d5, d6, d7)
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);

void setup() {
    // set up the LCD's number of columns and rows:
    lcd.begin(16, 2);
    // Print a message to the LCD.
    lcd.print("hello, world!");
}

void loop() {
    // set the cursor to column 0, line 1
    // (note: line 1 is the second row, since counting begins with 0):
    lcd.setCursor(0, 1);
    // print the number of seconds since reset:
    lcd.print(millis() / 1000);
    delay(100);
}
```


LCD

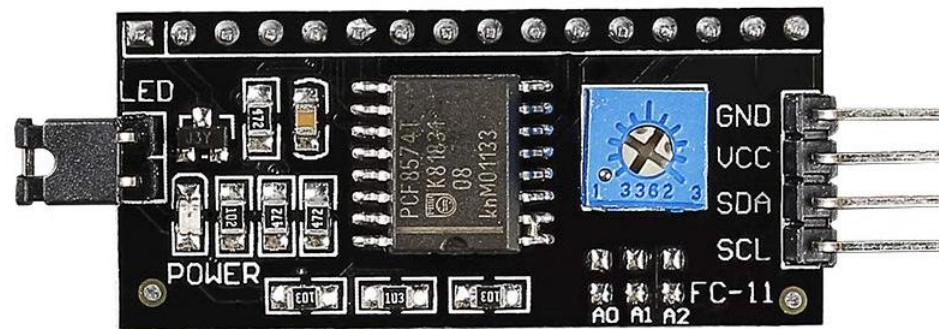
I2C bus

I2C is used for communication between Arduino and LCD.

I2C has two wires:

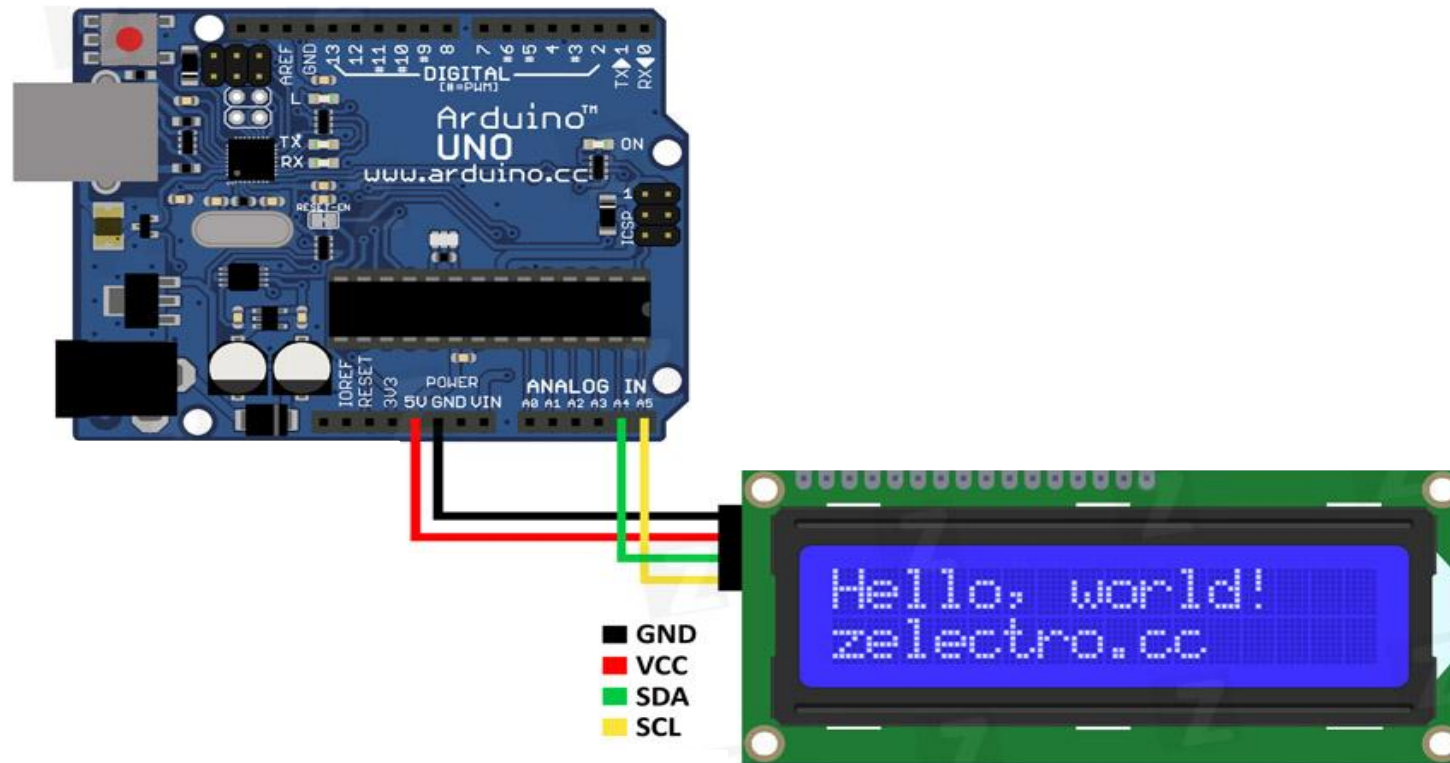
- Serial data (SDA): also connected to A4 pin
- Serial clock (SCL): also connected to A5 pin

The LCD screen contrast is controlled using an I2C buspotentiometer.



LCD

Connections



LCD

Code

```
#include <LiquidCrystal_I2C.h>

int I2Caddress = 0x27;      // I2C address of I2C bus
int LCDcol = 16;           // number of LCD columns
int LCDrow = 2;            // number of LCD rows
LiquidCrystal_I2C lcd(I2Caddress,LCDcol,LCDrow); // I2C address and LCD size

void setup() {
  lcd.init(); // initialise LCD
  lcd.backlight();
  lcd.setCursor(0,0);      // move cursor to column 1 row 1
  lcd.print("Hello World!"); // print "Hello World!" to LCD
}

void loop() {
}
```

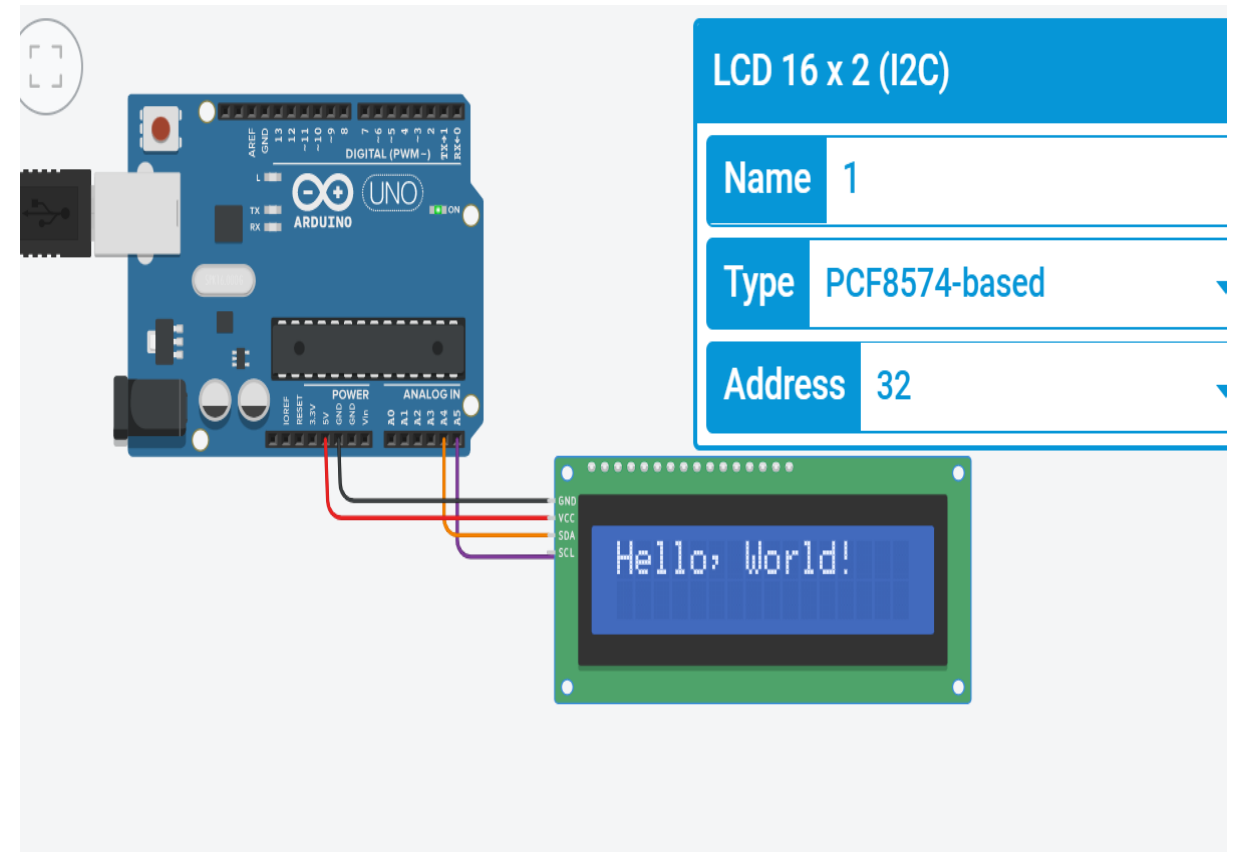
The backlight can be controlled using the following instructions:

- `lcd.backlight();` // turns on the backlight
- `lcd.noBacklight();` // turns off the backlight

LCD

Tinkercad simulation

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(32,16,2);
int t=3000;
void setup() {
  lcd.begin(16,2);
  lcd.init();
  lcd.backlight();
}
void loop() {
  lcd.setCursor(0,0);
  lcd.print("Hello, World!");
  delay(t);
  lcd.clear();
}
```

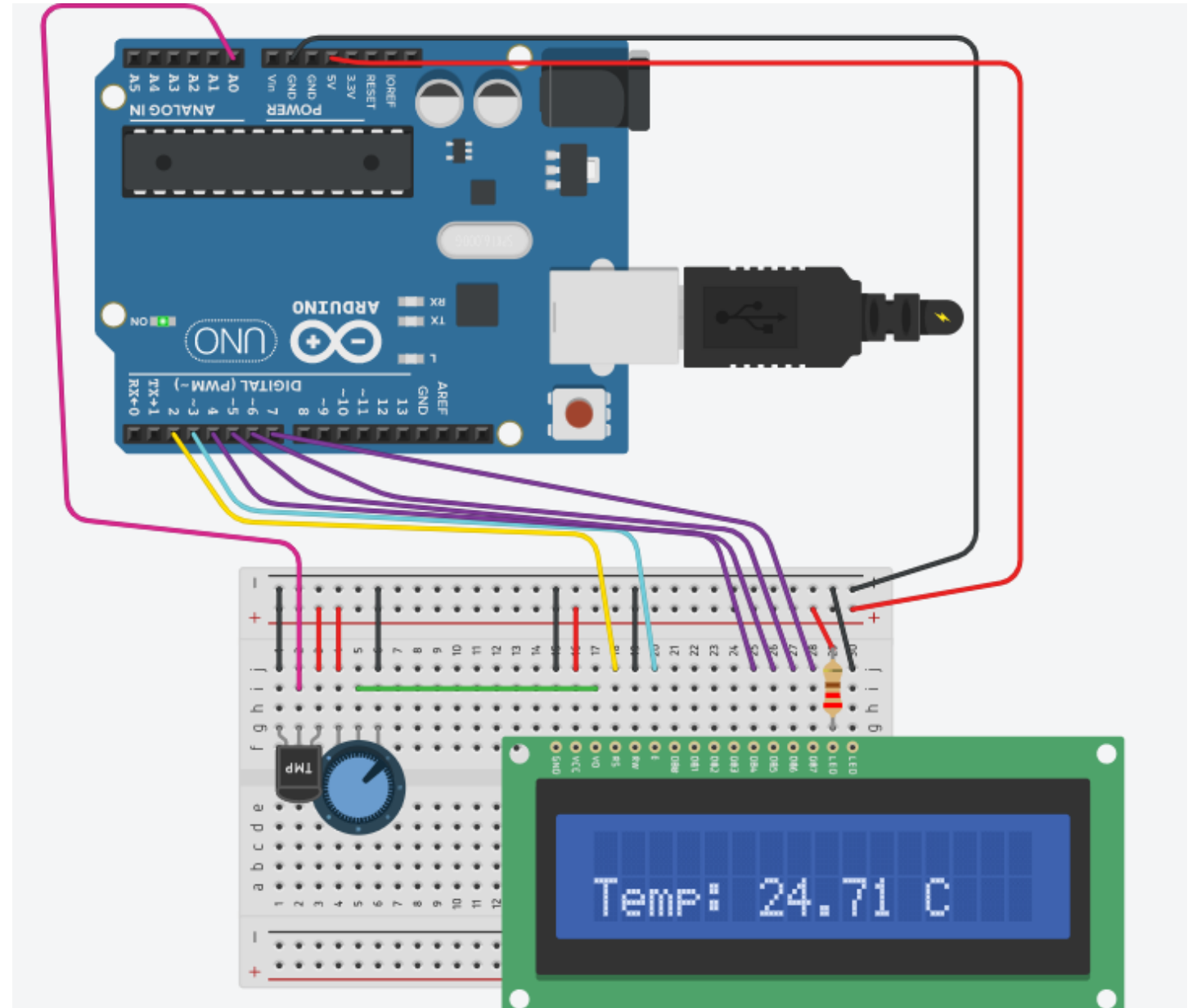


LCD

Exercise

Write an Arduino program that reads the temperature using the temperature sensor and displays the temperature Value on the LCD.

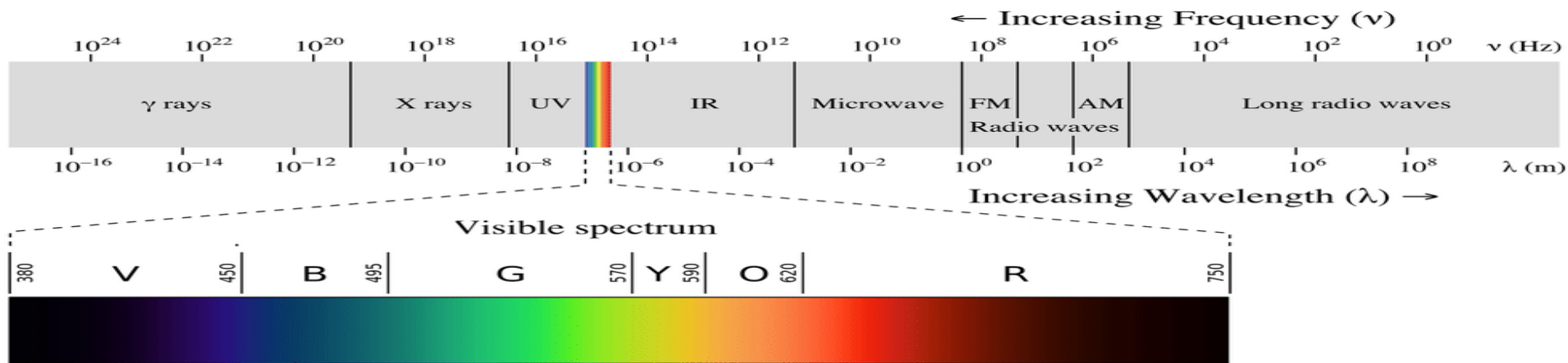
1. Using normal LCD
2. Using LCD_I2C



IR REMOTE

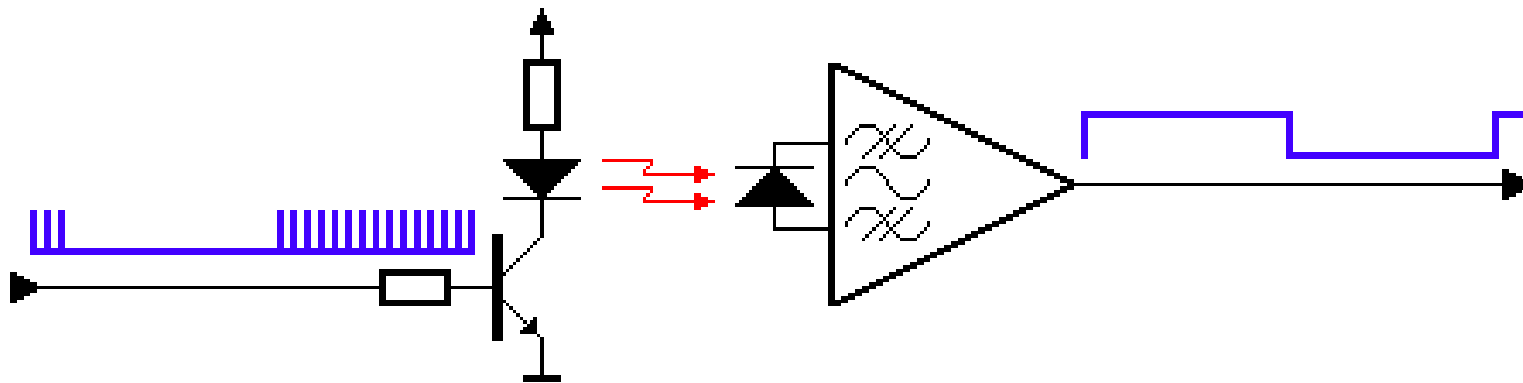
IR REMOTE

- An IR remote and receiver communicate with each other by transmitting and decoding a signal in the form of pulsed IR radiation
- Infrared radiation (IR), or infrared light, is a type of electromagnetic radiation with wavelengths ranging from 700 nm to 1 mm.
- IR radiation is invisible to the human eye.
- Since IR transmission is a wireless protocol based on a type of light, it requires a clear line of sight between the transmitter (the remote) and the receiver. This means it can't transmit through walls or ceilings, unlike WiFi or Bluetooth.



IR REMOTE

- Unfortunately, the IR LED in your remote is not the only source of IR radiation.
- Any object that has a temperature also radiates in the infrared spectrum.
- This phenomenon is also used by thermal cameras to detect heat.
- All this ambient IR can interfere with the communication between the remote and the receiver.
- So how does the receiver only detect the IR signal coming from the remote?
- With signal modulation, the IR light source at the end of the remote is blinked with a specific frequency.
- In consumer electronics, this carrier frequency is usually around 38 kHz.
- The receiver is built in such a way that it only lets IR through that is coming in at 38 kHz.



IR REMOTE

Types of IR receivers

IR receivers, called IR sensors or IR detection diodes, usually come in two different form factors. You can either buy the diodes separately or mounted on a board.

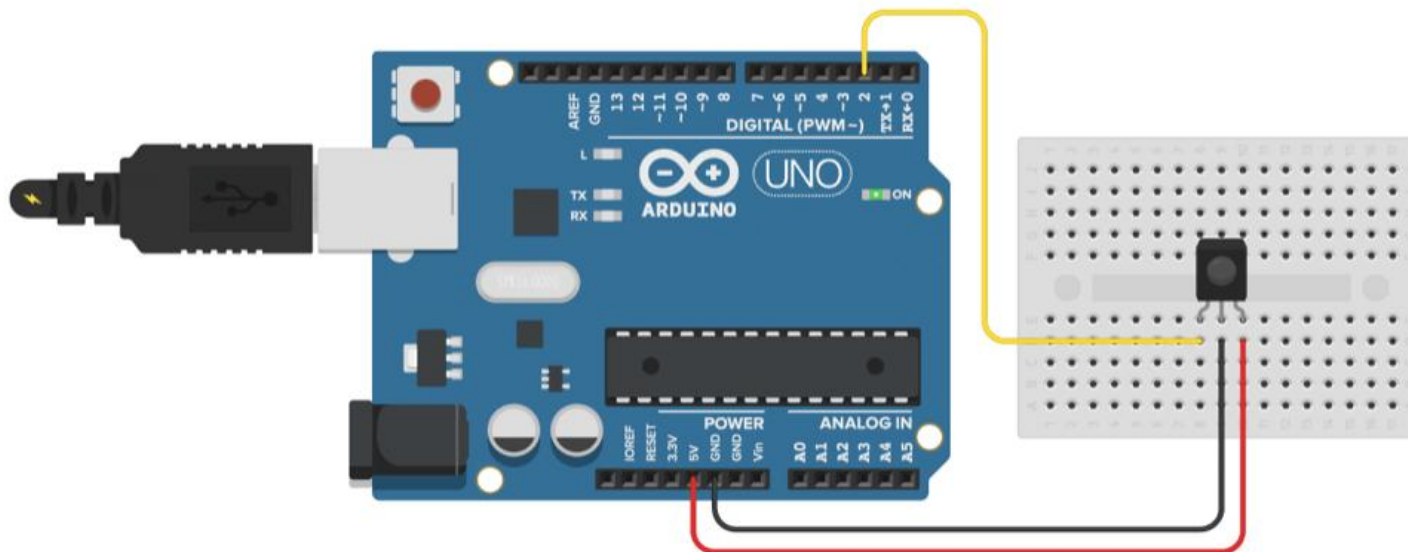


IR REMOTE

Connections

An IR receiver has three wires:

- The output wire can be connected to any of the digital pins of the Arduino.
- Vcc
- GND



IR REMOTE

Code

- There are many different types of remotes on the market (different number of keys and values printed on the keys), therefore, we need to determine which received signal corresponds to which key.
- The IR remote library will read the signal and output a specific code in the form of a decimal number depending on which key is pressed.
- By printing this output in the Serial Monitor, we can create a conversion table.

```
#include <IRremote.h> //include the remote control library
#define IR_PIN 2
IRrecv irReceiver(IR_PIN);
```

```
void setup(){
  Serial.begin(9600); // Begin serial comm.
  irReceiver.begin(IR_PIN, ENABLE_LED_FEEDBACK); /* Creating and starting receiver object with led13 blink on */
}
```

```
void loop(){
  if (irReceiver.decode()){
    Serial.print("Code for pressed button: ");
    Serial.println(irReceiver.decodedIRData.command);
    irReceiver.resume();
  }
}
```



Serial Monitor

```
Code for pressed button: 12
Code for pressed button: 16
Code for pressed button: 17
Code for pressed button: 18
```

IR REMOTE



Button	Code	Button	Code
POWER	0	1	16
VOL+	1	2	17
FUNC/STOP	2	3	18
<<	4	4	20
>	5	5	21
>>	6	6	22
▼	8	7	24
VOL-	9	8	25
▲	10	9	26
0	12		
EQ	13		
ST/REPT	14		

IR REMOTE

Exercise

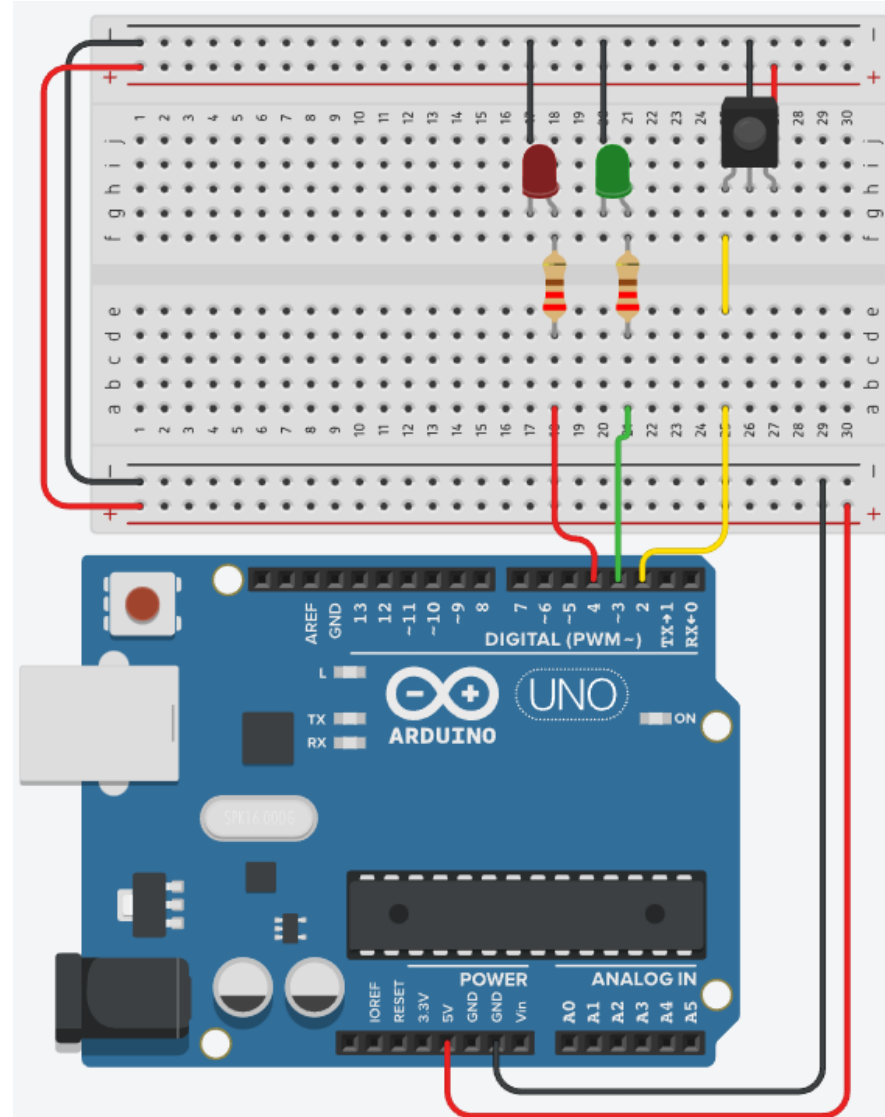
Reads a key (password) from an IR Remote Control

If entered key is 420:

Turn green led on

Otherwise:

Turn green led off



IR REMOTE

Exercise

```
#include <IRremote.h> //including infrared remote header file
int IR_PIN = 2; // the pin where you connect the output pin of IR sensor
IRrecv irReceiver(IR_PIN);

int greenLed = 3;
int redLed = 4;
String IR_Input;
String key = "420";

void setup()
{
  Serial.begin(9600);
  irReceiver.begin(IR_PIN, ENABLE_LED_FEEDBACK);
  pinMode(greenLed, OUTPUT);
  pinMode(redLed, OUTPUT);
}
```

IR REMOTE

Exercise

```
void loop(){
  if (irReceiver.decode()){
    if(irReceiver.decodedIRData.command == 12)
      IR_Input += '0';
    else if(irReceiver.decodedIRData.command == 16)
      IR_Input += '1';
    else if(irReceiver.decodedIRData.command == 17)
      IR_Input += '2';
    else if(irReceiver.decodedIRData.command == 18)
      IR_Input += '3';
    else if(irReceiver.decodedIRData.command == 20)
      IR_Input += '4';
    else if(irReceiver.decodedIRData.command == 21)
      IR_Input += '5';
    else if(irReceiver.decodedIRData.command == 22)
      IR_Input += '6';
    else if(irReceiver.decodedIRData.command == 24)
      IR_Input += '7';
    else if(irReceiver.decodedIRData.command == 25)
      IR_Input += '8';
    else if(irReceiver.decodedIRData.command == 26)
      IR_Input += '9';
    else if(irReceiver.decodedIRData.command == 0)
      IR_Input = "";
    Serial.print("IR remote input so far: ");
    Serial.println(IR_Input);
    irReceiver.resume(); // Receive the next value
  }
}
```

IR REMOTE

Exercise

```
if(IR_Input == key){  
    digitalWrite(greenLed, HIGH);  
    digitalWrite(redLed, LOW);  
}  
else{  
    digitalWrite(greenLed, LOW);  
    digitalWrite(redLed, HIGH);  
}  
}
```